

CHAPTER 2

Simple Neural Nets For Pattern Classification

General Discussion

- ❑ One of the simplest tasks that neural nets can be trained to perform is **pattern classification**.
- ❑ In pattern classification problems, each input vector (pattern) belongs, or does not belong, to a particular **class** or category.
- ❑ For a neural net approach, we assume we have a set of **training patterns** for which the correct classification is known.
- ❑ The output unit represents membership in the class with a response **of 1; a response of - 1** (or 0 if binary representation is used) indicates that the pattern is not a member of the class.

General Discussion

- ❑ In 1963, neural networks was used to detect heart abnormalities with EKG types of data as input (46 measurements) and classified them into "normal" or "abnormal".
- ❑ When patterns may or may not belong to several classes, there is an output unit for each class.
- ❑ In this chapter, we shall discuss three methods of **training** a simple single layer neural net for pattern classification: the **Hebb rule**, the **perceptron** learning rule, and the **delta rule**.

Architecture

- ❑ The basic architecture of the simplest possible neural networks that perform pattern classification consists of a layer of **input units (as many units as the patterns to be classified have components)** and a single output unit.

Architecture

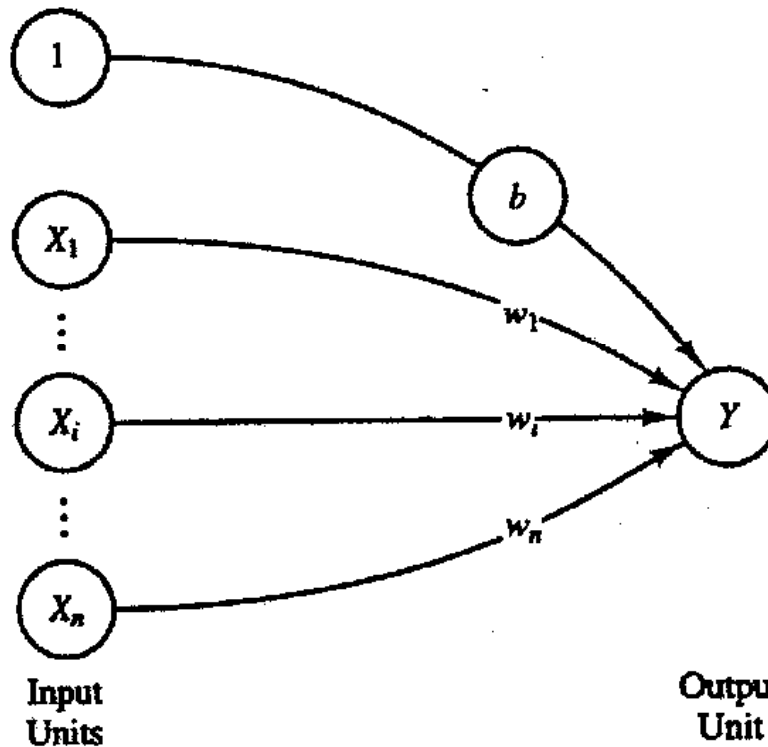


Figure 2.1 Single-layer net for pattern classification.

Biases and Thresholds

- ❑ A bias acts exactly as a weight on a connection from a unit whose activation is always 1.
- ❑ Increasing the bias increases the net input to the unit.
- ❑ If a bias is included, the activation function is typically taken to be:

$$f(\text{net}) = \begin{cases} 1 & \text{if net} \geq 0; \\ -1 & \text{if net} < 0; \end{cases}$$

$$\text{net} = b + \sum_i x_i w_i.$$

Biases and Thresholds

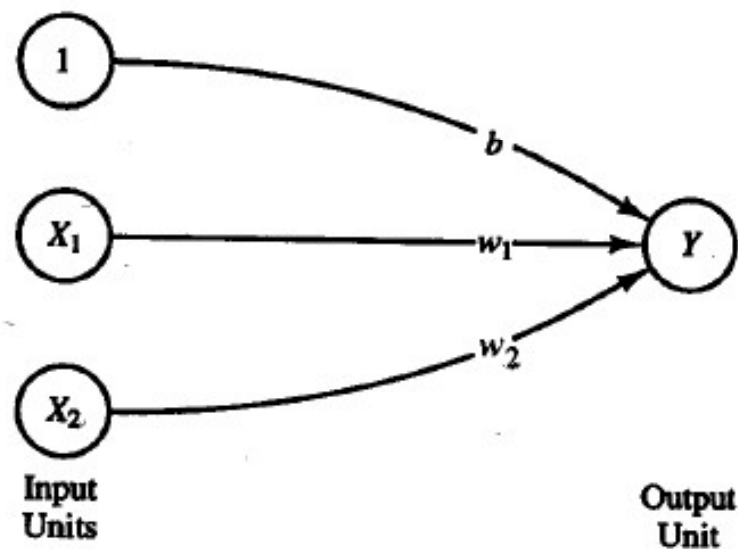
- ❑ Some authors do not use a bias weight, but instead use a fixed threshold θ for the activation function. In that case :

$$f(\text{net}) = \begin{cases} 1 & \text{if } \text{net} \geq \theta; \\ -1 & \text{if } \text{net} < \theta; \end{cases}$$

$$\text{net} = \sum_i x_i w_i.$$

The role of a bias and threshold

- we consider the separation of the input space into regions where the response of the net is positive and regions where the response is negative.



The role of a bias and threshold

- ❑ The boundary between the values of x_1 and x_2 :
- ❑ With bias:

$$b + x_1 w_1 + x_2 w_2 = 0,$$

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}.$$

- ❑ With threshold:

$$x_1 w_1 + x_2 w_2 = \theta,$$

$$x_2 = -\frac{w_1}{w_2} x_1 + \frac{\theta}{w_2}.$$

The role of a bias and threshold

- ❑ During training, values of w , and w_2 are determined so that the net will have the correct response for the training data.
- ❑ including neither a bias nor a threshold is equivalent to requiring the separating line (or plane or hyperplane for inputs with more components) to pass through the origin.

Linear Separability

- ❑ For a particular output unit, the desired response is a "yes" if the input pattern is a member of its class and a "no" if it is not.
- ❑ A "yes" response is represented by an output signal of 1, a "no" by an output signal of - 1 (for bipolar signals).
- ❑ Since the net input to the output unit is:

$$y_{in} = b + \sum_i x_i w_i,$$

- ❑ the boundary between the region is:

$$b + \sum_i x_i w_i = 0.$$

Linear Separability

- If there are weights (and a bias) so that all of the training input vectors for which the correct response is +1 lie on one side of the decision boundary and all of the training input vectors for which the correct response is -1 lie on the other side of the decision boundary, we say that the problem is "linearly separable."

Linear Separability

- ❑ Minsky and Papert [1988] showed that a **single-layer net can learn only linearly separable problems**.
- ❑ Furthermore, it is easy to extend this result to show that multilayer nets with linear activation functions are no more powerful than single-layer nets (since the composition of linear functions is linear).

AND function

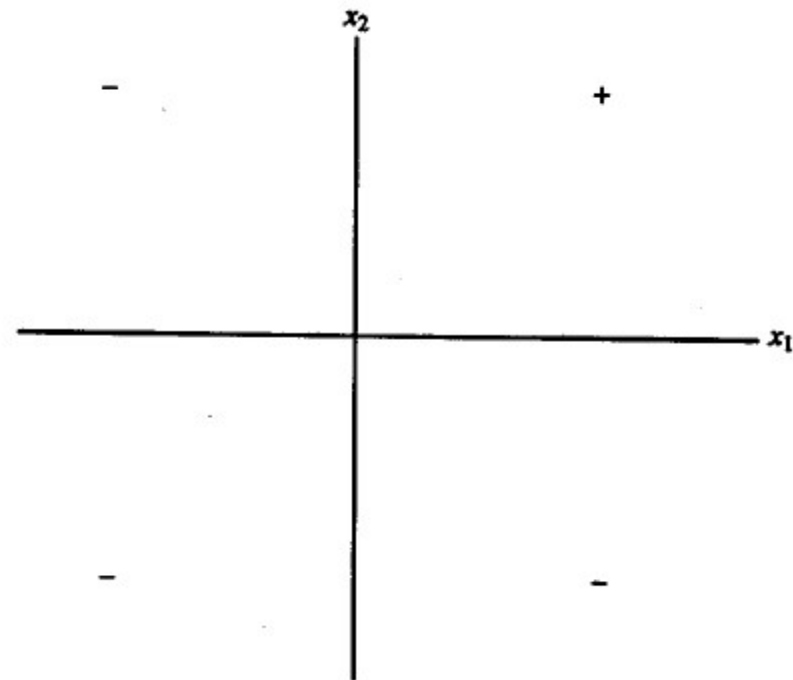
INPUT (x_1, x_2)	OUTPUT (t)
----------------------	----------------

(1, 1)	+1
--------	----

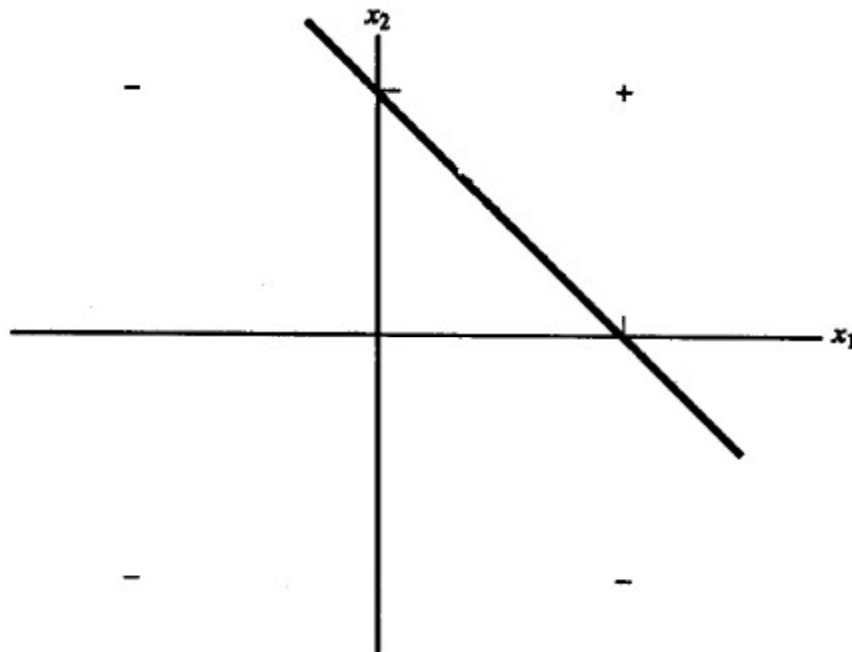
(1, -1)	-1
---------	----

(-1, 1)	-1
---------	----

(-1, -1)	-1
----------	----



AND function



$$x_2 = -x_1 + 1,$$

$$b = -1,$$

$$w_1 = 1,$$

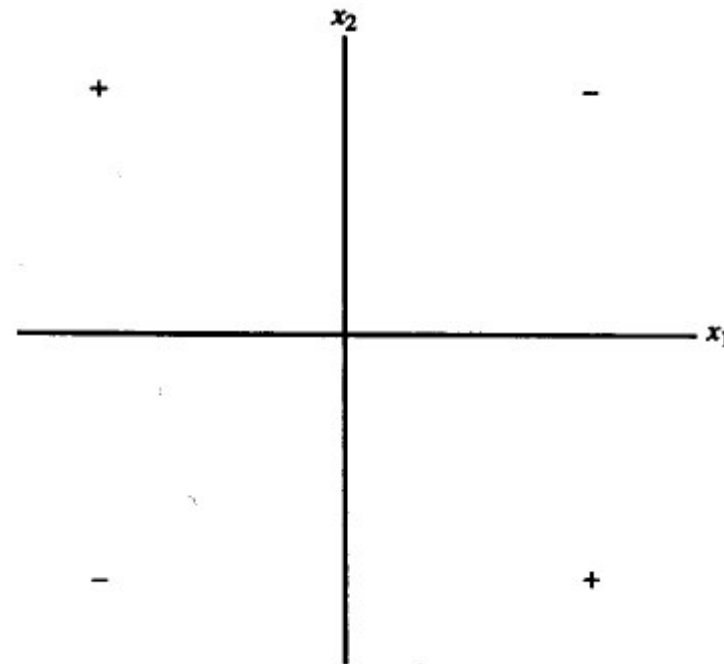
$$w_2 = 1.$$

Linear Separability

- ❑ OR function is also similar to AND function.
- ❑ The equations of the decision boundaries are not **unique**.
- ❑ Note that if a bias weight were not included in these examples, the decision boundary would be forced to go through the origin.
- ❑ Not all simple two-input, single-output mappings can be solved by a single layer net (even with a bias included), as is illustrated in Example 2.4.

XOR function

INPUT (x_1, x_2)	OUTPUT (t)
(1, 1)	-1
(1, -1)	+1
(-1, 1)	+1
(-1, -1)	-1



Data Representation

- ❑ Binary or Bipolar
- ❑ Binary representation is also not as good as bipolar if we want the net to generalize (i.e., respond to input data similar, but not identical to, training data).
- ❑ Using bipolar input, missing data can be distinguished from mistaken data. Missing values can be represented by "0" and mistakes by reversing the input value from + 1 to - 1, or vice versa.
- ❑ In general, bipolar representation is preferable

HEBB NET

- ❑ The earliest and simplest learning rule for a neural net is generally known as the Hebb rule.
- ❑ Hebb proposed that learning occurs by modification of the synapse strengths (weights) in a manner such that ***if two interconnected neurons are both "on" at the same time, then the weight between those neurons should be increased.***
- ❑ If data are represented in bipolar form, it is easy to express the desired weight update as:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y.$$

HEBB NET

- ❑ If the data are binary, this formula does not distinguish between a training pair in which an input unit is "on" and the target value is "off" and a training pair in which both the input unit and the target value are "off."

Algorithm

Step 0. Initialize all weights:

$$w_i = 0 \quad (i = 1 \text{ to } n).$$

Step 1. For each input training vector and target output pair, $s : t$, do steps 2–4.

Step 2. Set activations for input units:

$$x_i = s_i \quad (i = 1 \text{ to } n).$$

Step 3. Set activation for output unit:

$$y = t.$$

Step 4. Adjust the weights for

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad (i = 1 \text{ to } n).$$

Adjust the bias:

$$b(\text{new}) = b(\text{old}) + y.$$

Example 2.5

- AND function, **binary** input, **binary** targets

INPUT			TARGET
x_1	x_2	b	
1	1	1	1
1	0	1	0
0	1	1	0
0	0	1	0

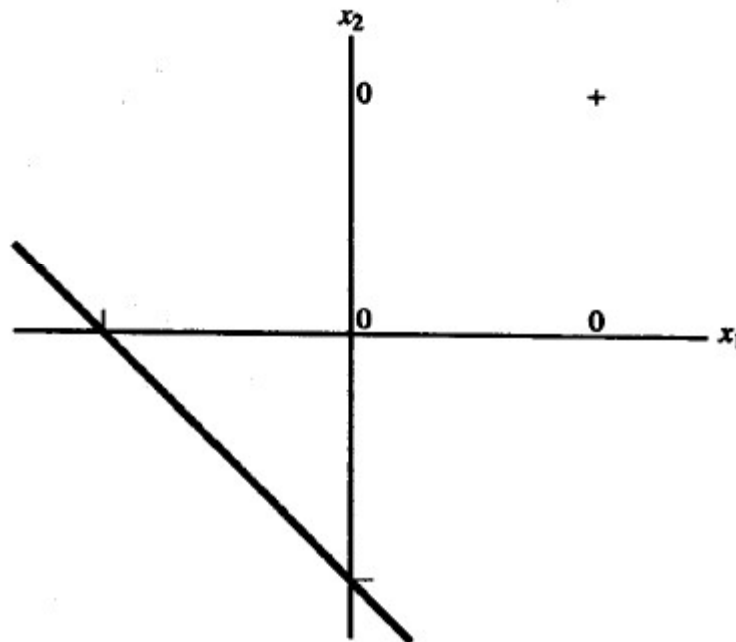
$$\Delta w_1 = x_1 t, \quad \Delta w_2 = x_2 t, \quad \Delta b = t.$$

Example 2.5

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
(1 0 1)	0	(0 0 0)	(1 1 1)
(0 1 1)	0	(0 0 0)	(1 1 1)
(0 0 1)	0	(0 0 0)	(1 1 1)

Example 2.5

$$x_2 = -x_1 - 1.$$



Example 2.7

- AND function, bipolar input, bipolar targets

INPUT			TARGET
$(x_1$	x_2	b	
(1	1	1)	1
(1	-1	1)	-1
(-1	1	1)	-1
(-1	-1	1)	-1

Example 2.7

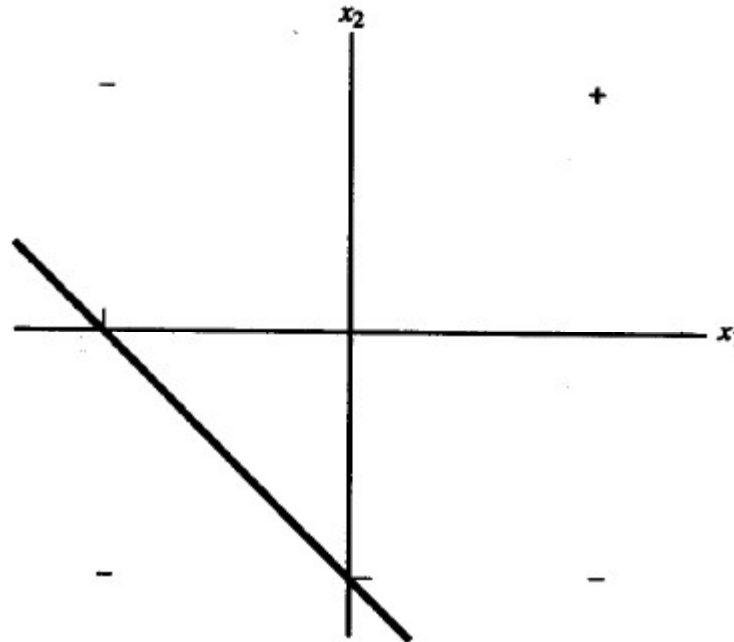
□ First input:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
$(0 \ 0 \ 1)$	1	$(1 \ 1 \ 1)$	$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$			$(1 \ 1 \ 1)$

Example 2.7

□ First input:

$$x_2 = -x_1 - 1.$$



Example 2.7

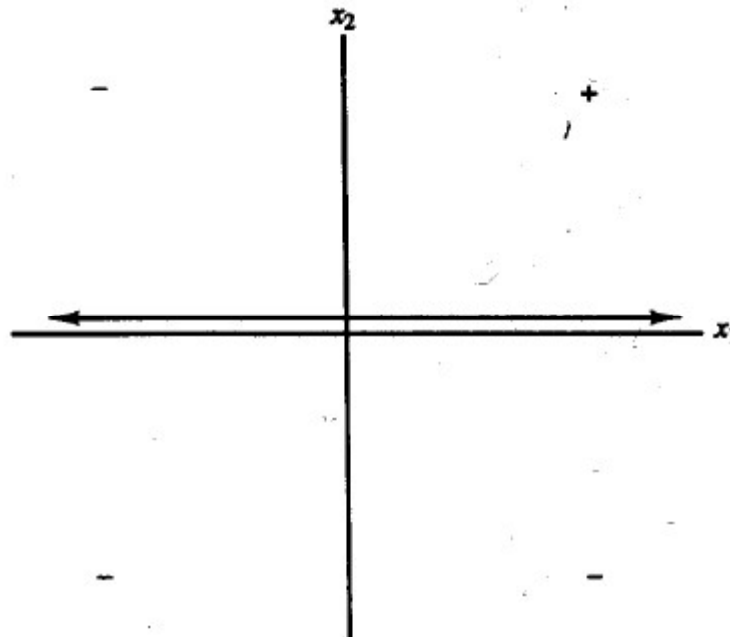
❑ Second input:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
$(1 \ -1 \ 1)$	-1	$(-1 \ 1 \ -1)$	$(1 \ 1 \ 1)$
			$(0 \ 2 \ 0)$

Example 2.7

□ Second input:

$$x_2 = 0.$$



Example 2.7

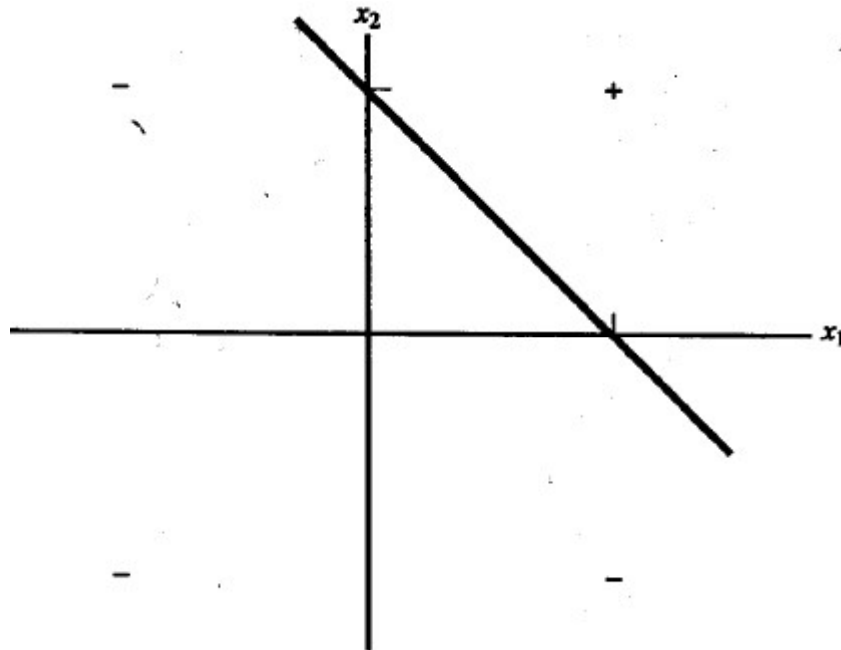
□ Third input:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$		$(\Delta w_1 \quad \Delta w_2 \quad \Delta b)$	$(w_1 \quad w_2 \quad b)$
$(-1 \quad 1 \quad 1)$	-1	$(1 \quad -1 \quad -1)$	$(0 \quad 2 \quad 0)$
			$(1 \quad 1 \quad -1)$

Example 2.7

□ Third input:

$$x_2 = -x_1 + 1.$$



Example 2.7

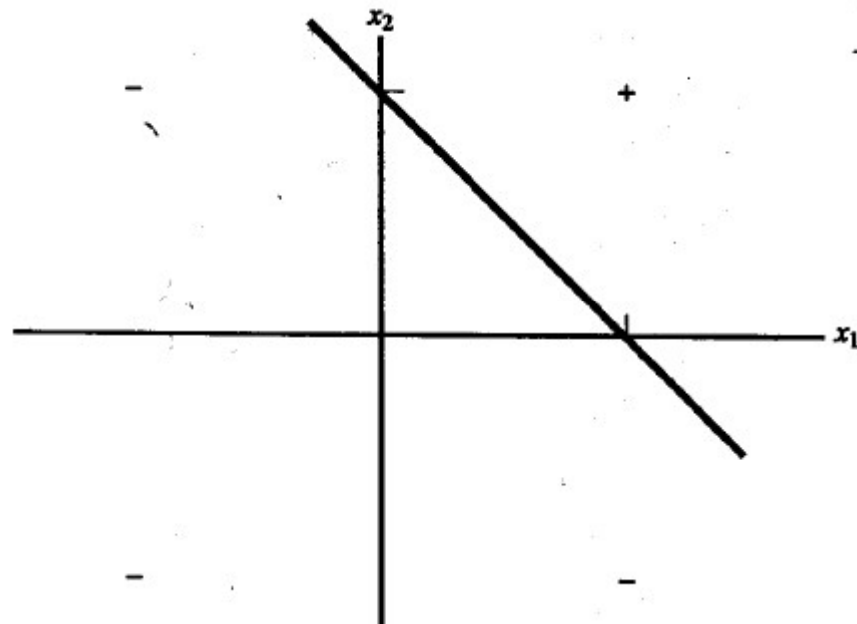
❑ Fourth input:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(1 \ 1 \ -1)$
$(-1 \ -1 \ 1)$	-1	$(1 \ 1 \ -1)$	$(2 \ 2 \ -2)$

Example 2.7

□ Fourth input :

$$x_2 = -x_1 + 1,$$



Example 2.8

□ Character recognition

```
# . . . #
. # . # .
. . # . .
. # . # .
# . . . #
```

Pattern 1

and

```
. # # # .
# . . . #
# . . . #
# . . . #
. # # # .
```

Pattern 2

1 -1 -1 -1 1, -1 1 -1 1 -1, -1 -1 1 -1 -1, -1 1 -1 1 -1,
1 -1 -1 -1 1,

and pattern 2 becomes

-1 1 1 1 -1, 1 -1 -1 -1 1, 1 -1 -1 -1 1, 1 -1 -1 -1 1, -1 1 1 1 -1,

Example 2.8

- ❑ The correct response for the first pattern is "on," or + 1, so the weights after presenting the first pattern are simply the input pattern.
- ❑ The bias weight after presenting this is + 1.
- ❑ The correct response for the second pattern is "off," or - 1, so the weight change when the second pattern is presented is
- ❑ 1 -1 -1 -1 1, -1 1 1 1 -1, -1 1 1 1 -1, -1 1 1 1 -1, 1 -1 -1 -1 1.

Example 2.8

- ❑ In addition, the weight change for the bias weight is -1.
- ❑ Adding the weight change to the weights representing the first pattern gives
- ❑ the final weights: 2 -2 -2 -2 2, -2 2 0 2 -2, -2 2 0 2 -2, -2 2 0 2 -2, -2 2 0 2 -2, 2 -2 -2 -2 2
- ❑ The bias weight is 0

Example 2.8

- ❑ The net input (for any input pattern) is the dot product of the input pattern with the weight vector.
- ❑ For the first training vector, the net input is 42, so the response is positive, as desired.
- ❑ For the second training pattern, the net input is -42, so the response is clearly negative, also as desired.

Example 2.8

- ❑ The first type of change is usually referred to as "mistakes in the data." changing a 1 to a - 1, or vice versa.
- ❑ The second type of change is called "missing data." the value 0, rather than 1 or - 1.
- ❑ In general, a net can handle more missing components than wrong components; in other words, with input data, "It's better not to guess."

Example 2.10

- Limitation of Hebb rule training for bipolar patterns.

INPUT				TARGET	WEIGHT CHANGE				WEIGHT			
x_1	x_2	x_3	1)		Δw_1	Δw_2	Δw_3	Δb	w_1	w_2	w_3	b
									(0	0	0	0)
(1	1	1	1)	1	(1	1	1	1)	(1	1	1	1)
(1	1	-1	1)	-1	(-1	-1	1	-1)	(0	0	2	0)
(1	-1	1	1)	-1	(-1	1	-1	-1)	(-1	1	1	-1)
(-1	1	1	1)	-1	(1	-1	-1	-1)	(0	0	0	-2)

Example 2.10

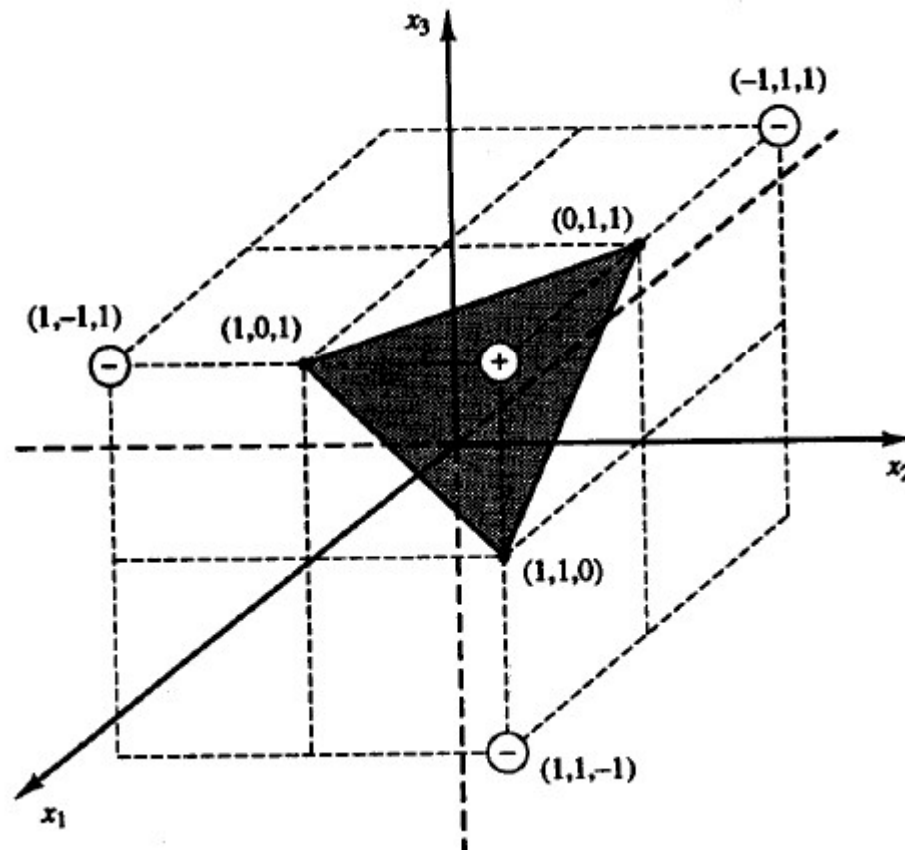
- Limitation of Hebb rule training for bipolar patterns.

INPUT				TARGET	WEIGHT CHANGE				WEIGHT			
x_1	x_2	x_3	1)		Δw_1	Δw_2	Δw_3	Δb	w_1	w_2	w_3	b
									(0	0	0	0)
(1	1	1	1)	1	(1	1	1	1)	(1	1	1	1)
(1	1	-1	1)	-1	(-1	-1	1	-1)	(0	0	2	0)
(1	-1	1	1)	-1	(-1	1	-1	-1)	(-1	1	1	-1)
(-1	1	1	1)	-1	(1	-1	-1	-1)	(0	0	0	-2)

Example 2.10

- ❑ Again, it is clear that the weights do not give the correct output for the first input pattern.
- ❑ Figure 2.13 shows that the input points are linearly separable; one possible plane, $x_1 + x_2 + x_3 + (-2) = 0$, to perform the separation is shown.
- ❑ This plane corresponds to a weight vector of (1 1 1) and a bias of -2.

Example 2.10



PERCEPTRON

- ❑ The perceptron learning rule is a more powerful learning rule than the Hebb rule.
- ❑ Under suitable assumptions, its iterative learning procedure can be proved to converge to the correct weights, i.e., the weights that allow the net to produce the correct output value for each of the training input patterns.

PERCEPTRON

- ❑ The activation function for each associator unit was the **binary** step function with an arbitrary, but fixed, threshold.

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

PERCEPTRON

- ❑ If an error occurred for a particular training input pattern, the weights would be changed according to the formula

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i,$$

- ❑ where the target value t is $+1$ or -1 and α is the learning rate.
- ❑ If an error did not occur, the weights would not be changed.
- ❑ Training would continue until no error occurred.

Architecture

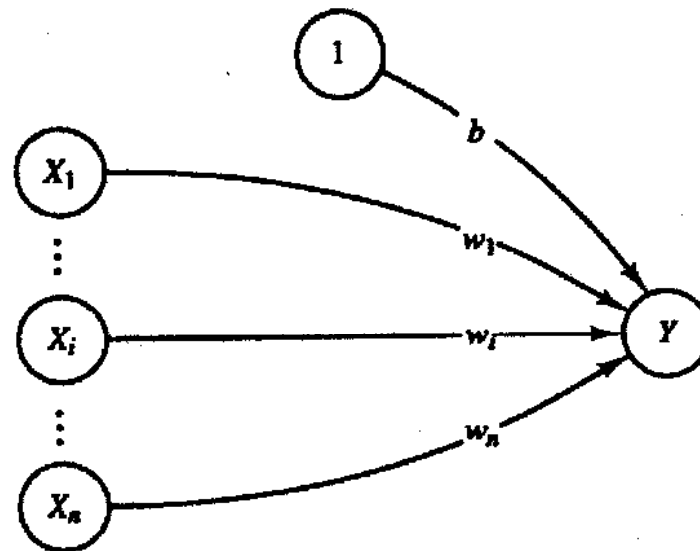


Figure 2.14 Perceptron to perform single classification.

Algorithm

- Step 0.* Initialize weights and bias.
(For simplicity, set weights and bias to zero.)
Set learning rate α ($0 < \alpha \leq 1$).
(For simplicity, α can be set to 1.)
- Step 1.* While stopping condition is false, do Steps 2–6.
- Step 2.* For each training pair $s:t$, do Steps 3–5.
- Step 3.* Set activations of input units:
$$x_i = s_i.$$
- Step 4.* Compute response of output unit:
$$y_{in} = b + \sum_i x_i w_i;$$

$$y = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Algorithm

Step 5. Update weights and bias if an error occurred for this pattern.

If $y \neq t$,

$$w_i(\text{new}) = w_i(\text{old}) + \alpha x_i,$$

$$b(\text{new}) = b(\text{old}) + \alpha t.$$

else

$$w_i(\text{new}) = w_i(\text{old}),$$

$$b(\text{new}) = b(\text{old}).$$

Step 6. Test stopping condition:
If no weights changed in Step 2, stop; else, continue.

Algorithm

- Two separating lines:

$$w_1x_1 + w_2x_2 + b > \theta,$$

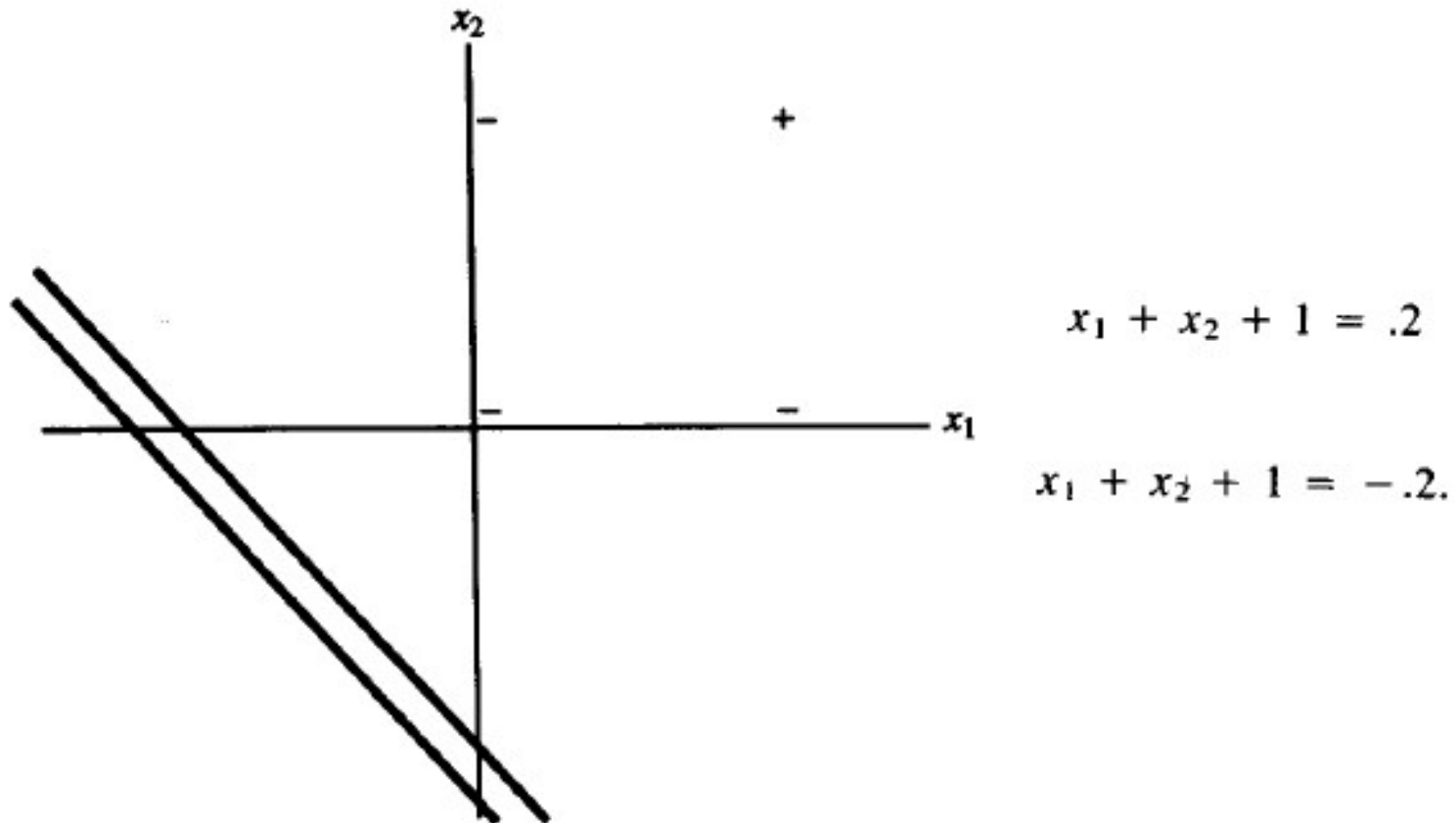
$$w_1x_1 + w_2x_2 + b < -\theta.$$

Example 2.11

- ❑ **AND** function: binary inputs, bipolar targets
- ❑ $\Theta = 0.2$
- ❑ First training sample:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS		
$(x_1$	x_2	$1)$					$(w_1$	w_2	$b)$
(0	0	0)	0	0	0	(0 0 0)	(0	0	0)
(1	1	1)	0	0	1	(1 1 1)	(1	1	1)

Example 2.11

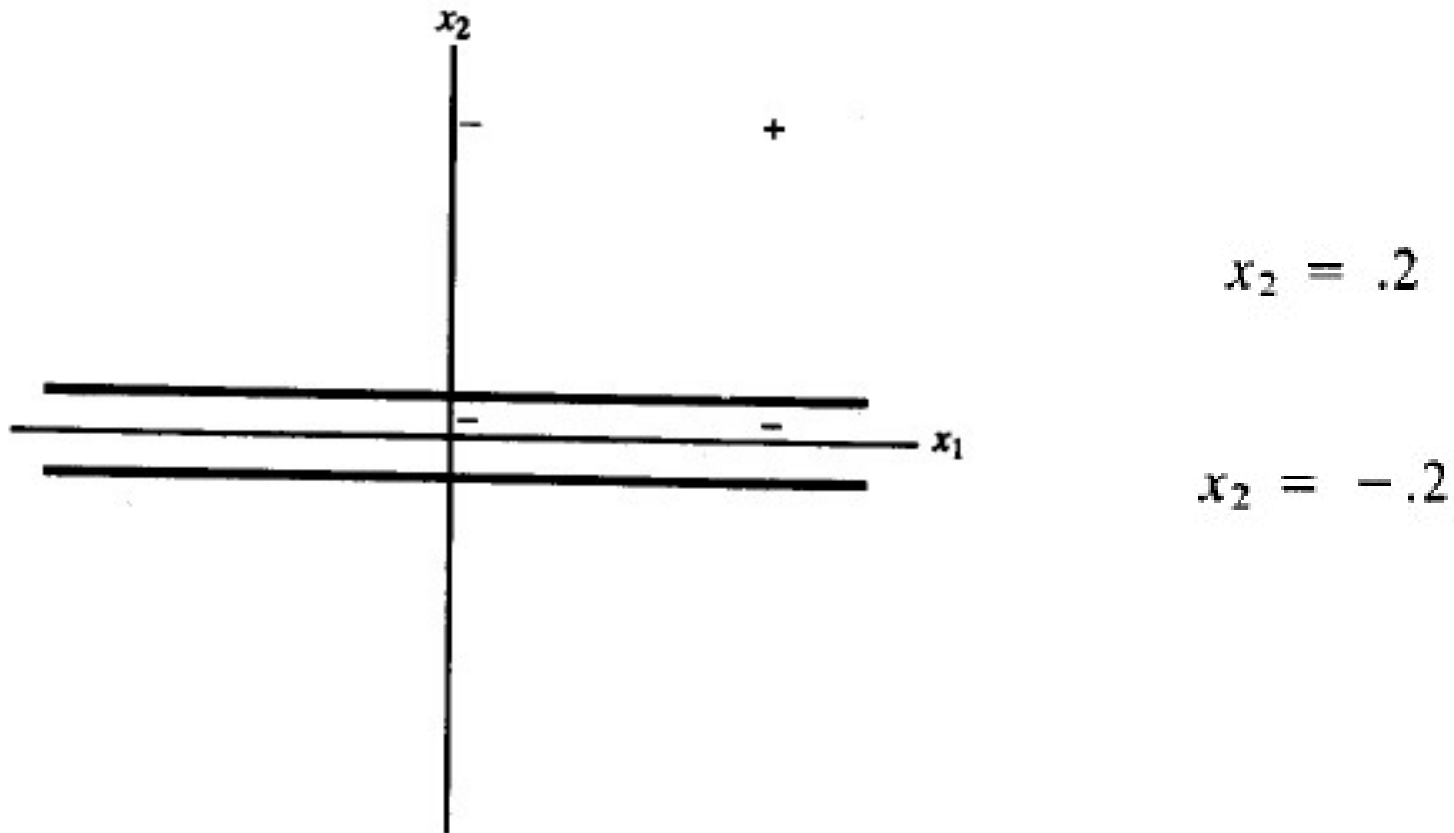


Example 2.11

❑ Second training sample:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES			WEIGHTS		
$(x_1$	x_2	$1)$							$(w_1$	w_2	$b)$
$(1$	0	$1)$	2	1	-1	$(-1$	0	$-1)$	$(0$	$\textcircled{1}$	$0)$

Example 2.11



Example 2.11

❑ Third training sample:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
					$(0 \quad 1 \quad 0)$
$(0 \quad 1 \quad 1)$	1	1	-1	$(0 \quad -1 \quad -1)$	$(0 \quad 0 \quad -1)$

❑ Fourth training sample:

INPUT	NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \quad x_2 \quad 1)$					$(w_1 \quad w_2 \quad b)$
					$(0 \quad 0 \quad -1)$
$(0 \quad 0 \quad 1)$	-1	-1	-1	$(0 \quad 0 \quad 0)$	$(0 \quad 0 \quad -1)$

Example 2.11

- ❑ The response of the all inputs are negative which is not true for the (1,1). So we need more iteration.
- ❑ After ninth epoch:

(1	1	1)	0	0	1	(1	1	1)	(3	3	-3)
(1	0	1)	0	0	-1	(-1	0	-1)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

- ❑ After tenth epoch:

(1	1	1)	1	1	1	(0	0	0)	(2	3	-4)
(1	0	1)	-2	-1	-1	(0	0	0)	(2	3	-4)
(0	1	1)	-1	-1	-1	(0	0	0)	(2	3	-4)
(0	0	1)	-4	-1	-1	(0	0	0)	(2	3	-4)

Example 2.11

□ Separating lines are:

$$2x_1 + 3x_2 - 4 > .2,$$

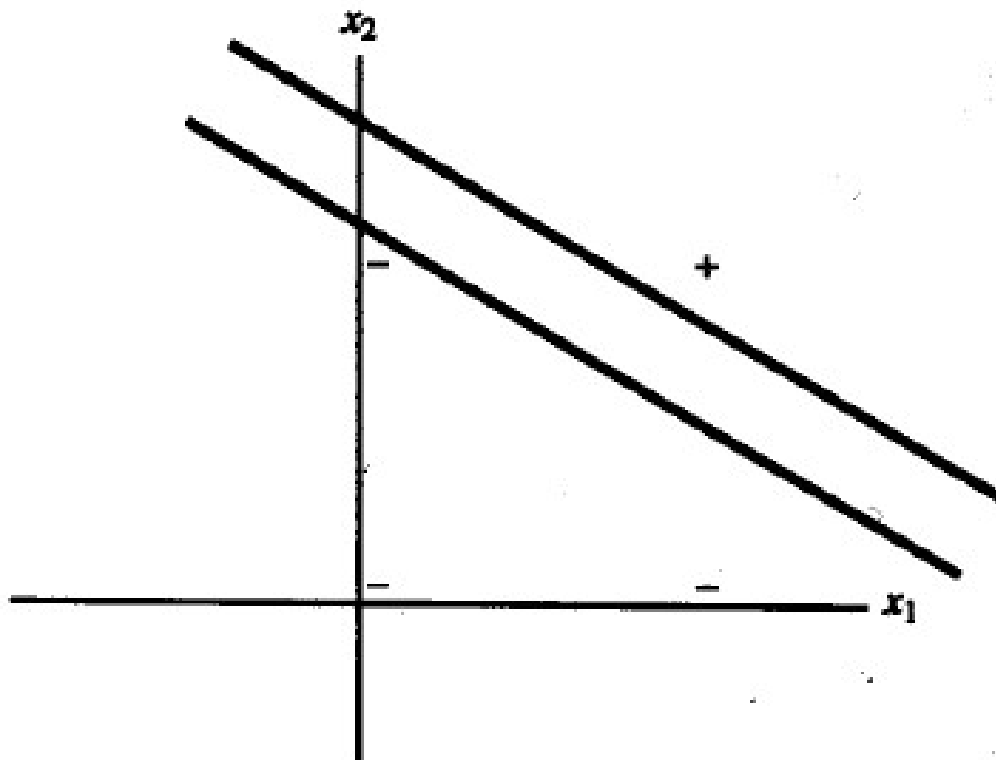
$$2x_1 + 3x_2 - 4 < -.2,$$

$$x_2 = -\frac{2}{3}x_1 + \frac{7}{5},$$

$$x_2 = -\frac{2}{3}x_1 + \frac{19}{15}$$

Example 2.11

□ Separating lines are:



Example 2.12

- ❑ A Perceptron for the AND function: bipolar inputs and targets.
- ❑ The training process $\text{Alpha}=1$ and threshold and initial weights = 0.

Example 2.12

□ First epoch:

INPUT			NET	OUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1$	x_2	$1)$					$(w_1$ w_2 $b)$
							$(0$ 0 $0)$
$(1$	1	$1)$	0	0	1	$(1$ 1 $1)$	$(1$ 1 $1)$
$(1$	-1	$1)$	1	1	-1	$(-1$ 1 $-1)$	$(0$ 2 $0)$
$(-1$	1	$1)$	2	1	-1	$(1$ -1 $-1)$	$(1$ 1 $-1)$
$(-1$	-1	$1)$	-3	-1	-1	$(0$ 0 $0)$	$(1$ 1 $-1)$

Example 2.12

❑ Second epoch:

(1	1	1)	1	1	1	(0	0	0)	(1	1	-1)
(1	-1	1)	-1	-1	-1	(0	0	0)	(1	1	-1)
(-1	1	1)	-1	-1	-1	(0	0	0)	(1	1	-1)
(-1	-1	1)	-3	-1	-1	(0	0	0)	(1	1	-1)

- ❑ the system was fully trained after the first epoch.
- ❑ Bipolar representation reduces number of epochs.

Application Procedure

Step 0. Apply training algorithm to set the weights.

Step 1. For each input vector $\mathbf{x}$ to be classified, do Steps 2–3.

Step 2. Set activations of input units.

Step 3. Compute response of output unit:

$$y_{in} = \sum_i x_i w_i;$$

$$y = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Training for several output neurons

Step 0. Initialize weights and biases
(0 or small random values).

Step 1. While stopping condition is false, do Steps 1–6.

Step 2. For each bipolar training pair $s : t$, do Steps 3–5.

Step 3. Set activation of each input unit, $i = 1, \dots, n$:

$$x_i = s_i.$$

Step 4. Compute activation of each output unit,
 $j = 1, \dots, m$:

$$y_in_j = b_j + \sum_i x_i w_{ij}.$$

$$y_j = \begin{cases} 1 & \text{if } y_in_j > \theta \\ 0 & \text{if } -\theta \leq y_in_j \leq \theta \\ -1 & \text{if } y_in_j < -\theta \end{cases}$$

Training for several output neurons

Step 5. Update biases and weights, $j = 1, \dots, m$;
 $i = 1, \dots, n$:

If $t_j \neq y_j$, then

$$b_j(\text{new}) = b_j(\text{old}) + t_j;$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + t_j x_i.$$

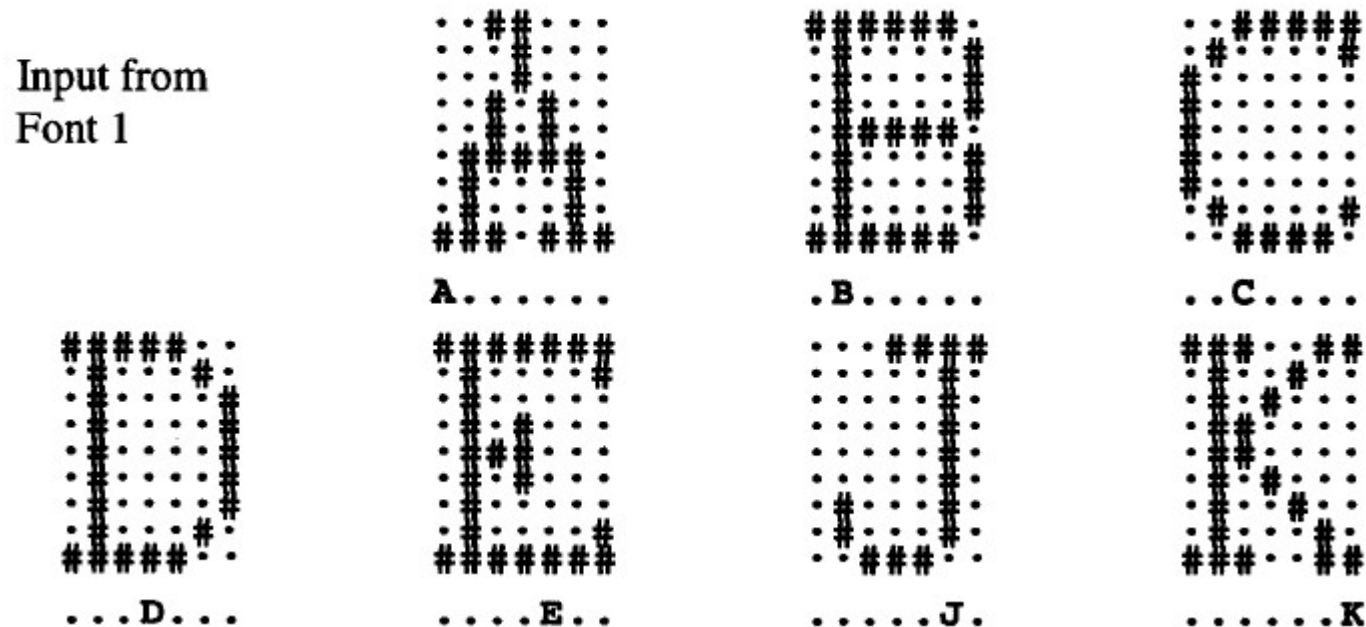
Else, biases and weights remain unchanged.

Step 6. Test for stopping condition:

If no weight changes occurred in Step 2, stop; otherwise, continue.

Example 2.15

- We want to classify the following 21 characters written by 3 fonts into 7 classes.



D

A.

#	#	#	#	#	#	#
#	.	.	.	.	.	.
#	.	.	.	.	.	.
#	#	#	#	.	.	.
#	.	.	.	.	.	.
#	.	.	.	.	.	.
#	#	#	#	#	#	#

. . . E . .

. # #
. # #
. # #
. # #
. # #
. #

.B....

.....J.

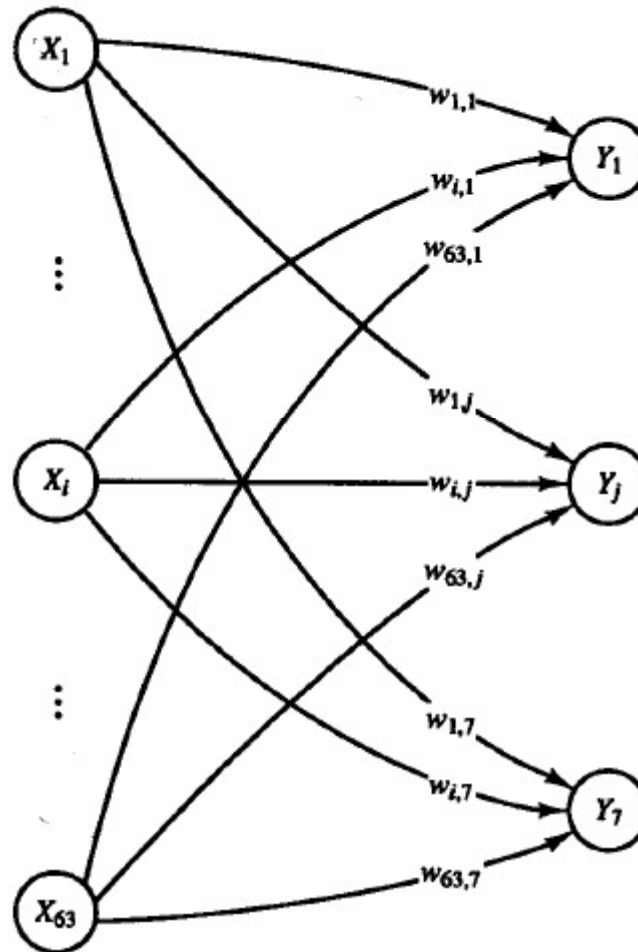
. . C . . .

[illegible]

.....R

C

Example 2.15



D

A.

...E..

.B

.....J.

C. E. N.

.....K

... D ...

[illegible][illegible][illegible]

. . . D . . .

• • • # • •
• • # • •
• • # • •
• • # • •
○ # ○ # ○

@ @ #

• • • • •

• • # • •
@ ○ # ○ #
• • # • •
• • # • •
○ # ○ # #
• • # #

. . . E . K

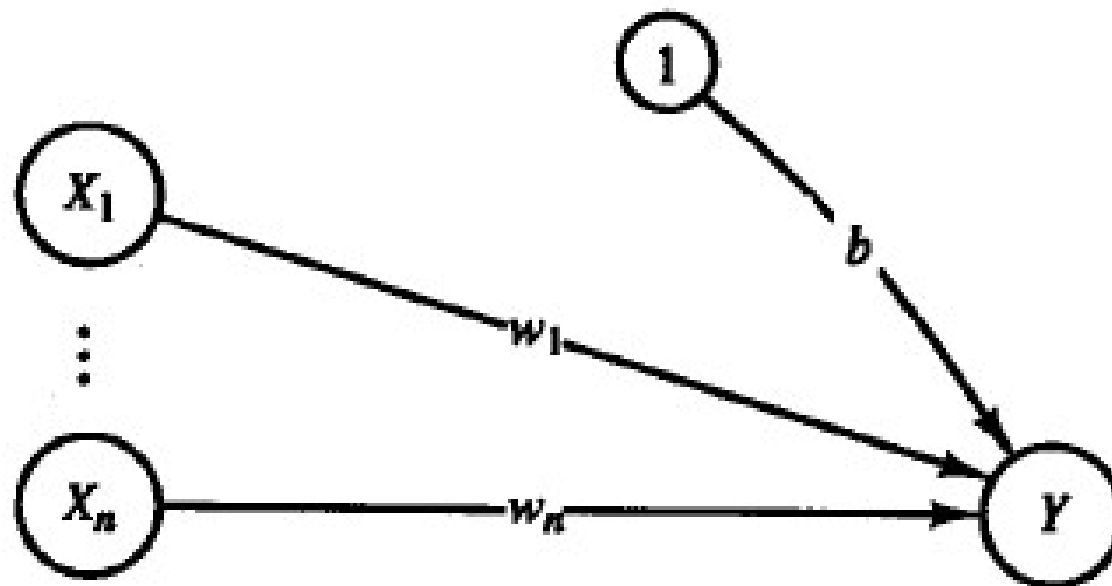
 @ # # # #
 . # # # #
 @ # # # #
 @ # # # #
 # # # # #
 . B
 . . . # # #
 @
 . . . # @
 #
 O
 #
 # . . . O
 . # # # .
 J .

[illegible]

Adaline

- ❑ The ADALINE = ADaptive Linear Neuron.
- ❑ an ADALINE can be trained using the **delta rule**, also known as the least mean squares (LMS) or Widrow-Hoff rule.
- ❑ During training, the activation of the unit is its net input, i.e., the activation function is the identity function.
- ❑ The learning rule minimizes the mean squared error between the activation and the target value.
- ❑

Architecture



Algorithm

- Step 0.** Initialize weights.
(Small random values are usually used.)
Set learning rate α .
(See comments following algorithm.)
- Step 1.** While stopping condition is false, do Steps 2–6.
- Step 2.** For each bipolar training pair $s:t$, do Steps 3–5.
- Step 3.** Set activations of input units, $i = 1 \dots n$:
- $$x_i = s_i.$$
- Step 4.** Compute net input to output unit:
- $$y_in = b + \sum_i x_i w_i.$$
- Step 5.** Update bias and weights, $i = 1 \dots n$:
- $$b(\text{new}) = b(\text{old}) + \alpha(t - y_in).$$
- $$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_in)x_i.$$
- Step 6.** Test for stopping condition:
If the largest weight change that occurred in Step 2 is smaller than a specified tolerance, then stop; otherwise continue.

Learning rate

- ❑ If Alpha is too large, the learning process will not converge; if too small a value is chosen, learning will be extremely slow
- ❑ For a single neuron, a practical range for the learning rate is: $0.1 \leq \alpha \leq 1.0$,
- ❑ N is the number of inputs.

Applications

Step 0. Initialize weights
(from ADALINE training algorithm given in Section 2.4.2).

Step 1. For each bipolar input vector $\mathbf{x}$, do Steps 2–4.

Step 2. Set activations of the input units to $\mathbf{x}$.

Step 3. Compute net input to output unit:

$$y_{in} = b + \sum_i x_i w_i.$$

Step 4. Apply the activation function:

$$y = \begin{cases} 1 & \text{if } y_{in} \geq 0; \\ -1 & \text{if } y_{in} < 0. \end{cases}$$

Example 2.16

- AND function: binary inputs, bipolar targets

x_1	x_2	t
1	1	1
1	0	-1
0	1	-1
0	0	-1

Example 2.16

- We want to minimize:

$$E = \sum_{p=1}^4 (x_1(p)w_1 + x_2(p)w_2 + w_0 - t(p))^2,$$

- Weights that minimize this error are:

$$w_1 = 1 \quad w_2 = 1, \quad w_0 = -\frac{3}{2}.$$

- Thus, the separating line is:

$$x_1 + x_2 - \frac{3}{2} = 0.$$

Example 2.16

- ❑ The total squared error for the four training patterns with these weights is 1.
- ❑ In Example 2.11, the boundary line of perceptron is:

$$x_2 = -\frac{2}{3}x_1 + \frac{4}{3}.$$

- ❑ The total squared error for the minimizing weights found by the perceptron is 10/9.

Example 2.17

- ❑ AND function: bipolar inputs, bipolar targets.
- ❑ The weights that minimize the total error for the bipolar form of the AND function are:

$$w_1 = \frac{1}{2} \quad w_2 = \frac{1}{2}, \quad w_0 = -\frac{1}{2}.$$

- ❑ Thus, the separating line is:

$$\frac{1}{2}x_1 + \frac{1}{2}x_2 - \frac{1}{2} = 0,$$

- ❑ as found by the perceptron in Example 2.12.

$$x_1 + x_2 - 1 = 0,$$

Derivation

- ❑ The delta rule changes the weights of the neural connections so as to minimize the difference between the net input to the output unit, y_{in} , and the target value t .
- ❑ The aim is to minimize the error over all training patterns.
- ❑ The delta rule for adjusting the I -th weight (for each pattern) is:

$$\Delta w_I = \alpha(t - y_{in})x_I.$$

Derivation

- ❑ The squared error for a particular training pattern is:

$$E = (t - y_{in})^2.$$

- ❑ The gradient of E is the vector consisting of the partial derivatives of E with respect to each of the weights.
- ❑ The gradient gives the direction of most rapid increase in E ; the opposite direction gives the most rapid decrease in the error.
- ❑ The error can be reduced by adjusting the weight w_i in the direction of $-\frac{\partial E}{\partial w_i}$

Derivation

$$\text{Since } y_{in} = \sum_{i=1}^n x_i w_I,$$

$$\begin{aligned} \frac{\partial E}{\partial w_I} &= -2(t - y_{in}) \frac{\partial y_{in}}{\partial w_I} \\ &= -2(t - y_{in})x_I. \end{aligned}$$

$$\Delta w_I = \alpha(t - y_{in})x_I.$$

Derivation

□ for more than one output unit:

$$\Delta w_{IJ} = \alpha(t_J - y_{in_J})x_I.$$

$$E = \sum_{j=1}^m (t_j - y_{in_j})^2.$$

$$\begin{aligned}\frac{\partial E}{\partial w_{IJ}} &= \frac{\partial}{\partial w_{IJ}} \sum_{j=1}^m (t_j - y_{in_j})^2 \\ &= \frac{\partial}{\partial w_{IJ}} (t_J - y_{in_J})^2,\end{aligned}$$

Derivation

$$y_{in_J} = \sum_{i=1}^n x_i w_{iJ},$$

$$\begin{aligned} \frac{\partial E}{\partial w_{IJ}} &= -2(t_J - y_{in_J}) \frac{\partial y_{in_J}}{\partial w_{IJ}} \\ &= -2(t_J - y_{in_J}) x_I. \end{aligned}$$

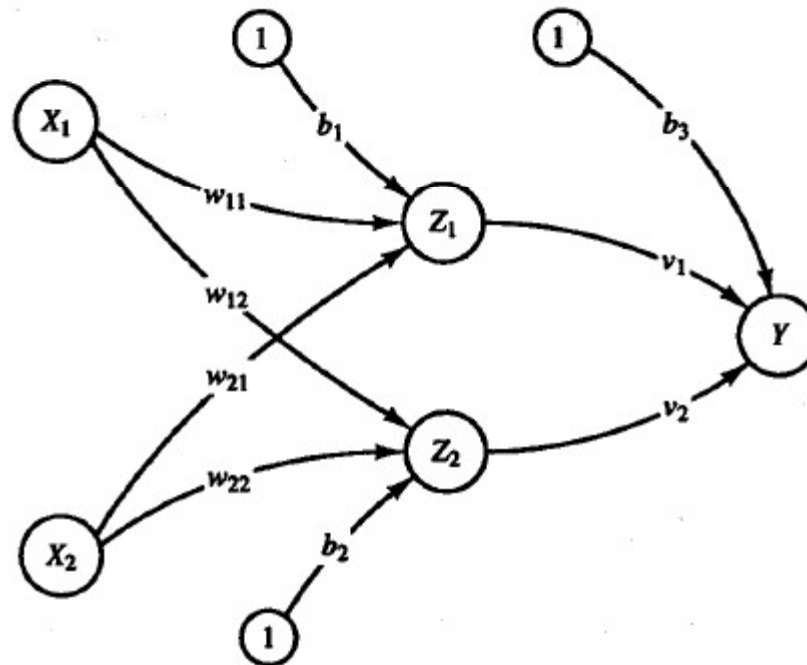
$$\Delta w_{IJ} = \alpha(t_J - y_{in_J}) x_I.$$

MADALINE

- ❑ a MADALINE consists of Many ADaptive Linear Neurons arranged in a multilayer net.

MADALINE

- ❑ a MADALINE consists of two hidden ADALINE and one output ADALINE unit.



Algorithm

- ❑ In the MRI algorithm (the original form of MADALINE training), only the weights for the hidden ADALINES are adjusted; the weights for the output unit are fixed.
- ❑ The MRII algorithm provides a method for adjusting all weights in the net.

MRI

- ❑ the weights v_1 and v_2 and the bias b_3 that feed into the output unit Y are determined so that the response of unit Y is 1 if the signal it receives from either Z_1 or Z_2 (or both) is 1 and is - 1 if both Z_1 and Z_2 send a signal of - 1.
- ❑ In other words, the unit Y performs the logic function OR on the signals it receives from Z_1 and Z_2 .
- ❑ The weights into Y are:

$$v_1 = \frac{1}{2} \quad v_2 = \frac{1}{2}, \quad b_3 = \frac{1}{2}$$

MRI

- ❑ The weights on the first hidden ADALINE () and the weights on the second hidden ADALINE () are adjusted according to the algorithm.
- ❑ The activation function for units Z1, Z2, and Y is:

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0; \\ -1 & \text{if } x < 0. \end{cases}$$

MRI

Step 0. Initialize weights:

Weights w_1 and w_2 and the bias b_3 are set as described;
small random values are usually used for ADALINE weights.

Set the learning rate α as in the ADALINE training algorithm (a small value).

Step 1. While stopping condition is false, do Steps 2–8.

Step 2. For each bipolar training pair, $s:t$, do Steps 3–7.

Step 3. Set activations of input units:

$$x_i = s_i.$$

Step 4. Compute net input to each hidden ADALINE unit:

$$z_{in1} = b_1 + x_1w_{11} + x_2w_{21},$$

$$z_{in2} = b_2 + x_1w_{12} + x_2w_{22}.$$

MRI

Step 5. Determine output of each hidden ADALINE unit:

$$z_1 = f(z_in_1),$$

$$z_2 = f(z_in_2).$$

Step 6. Determine output of net:

$$y_in = b_3 + z_1v_1 + z_2v_2;$$

$$y = f(y_in).$$

Step 7. Determine error and update weights:
If $t = y$, no weight updates are performed.
Otherwise:

If $t = 1$, then update weights on Z_J ,
the unit whose net input is closest to 0,

$$b_J(\text{new}) = b_J(\text{old}) + \alpha(1 - z_in_J),$$

$$w_{iJ}(\text{new}) = w_{iJ}(\text{old}) + \alpha(1 - z_in_J)x_i;$$

MRI

If $t = -1$, then update weights on all units Z_k that have positive net input,

$$b_k(\text{new}) = b_k(\text{old}) + \alpha(-1 - z_{in_k}),$$

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha(-1 - z_{in_k})x_i.$$

Step 8. Test stopping condition.
If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of weight update iterations (Step 2) have been performed, then stop; otherwise continue.

MRI

- ❑ Step 7 is motivated by the desire to (1) update the weights only if an error occurred and (2) update the weights in such a way that it is more likely for the net to produce the desired response.
- ❑ If $t = 1$ and error has occurred, it means that all Z units had value - 1 and at least one Z unit needs to have a value of + 1.
- ❑ Therefore, we change the Z unit whose net input is closest to 0 and adjust its weights.

MRI

- ❑ If $t = -1$ and error has occurred, it means that at least one Z unit had value $+1$ and all Z units must have value -1 .
- ❑ Therefore, we adjust the weights on all of the Z units with positive net input.

Example 2-20

- ❑ Training a MADALINE for the XOR function.
- ❑ We use MRI algorithm and only the first weight update is shown.
- ❑ Training patterns are:



x_1	x_2	t
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

Example 2-20

- Weight of Z_1 , Z_2 are small random values. Weight of Y is those found in Example 2.19 and Alpha is 0.5.

Weights into Z_1

w_{11}	w_{21}	b_1
.05	.2	.3

Weights into Z_2

w_{12}	w_{22}	b_2
.1	.2	.15

Weights into Y

v_1	v_2	b_3
.5	.5	.5

Example 2-20

□ *Step 1.* Begin training.

Step 2. For the first training pair, (1, 1): -1

Step 3. $x_1 = 1, \quad x_2 = 1$

Step 4. $z_{in1} = .3 + .05 + .2 = .55,$
 $z_{in2} = .15 + .1 + .2 = .45.$

Step 5. $z_1 = 1,$
 $z_2 = 1.$

Step 6. $y_{in} = .5 + .5 + .5;$
 $y = 1.$

Step 7. $t - y = -1 - 1 = -2 \neq 0$, so an error occurred.

Example 2-20



update the weights on unit Z_1 as follows:

$$\begin{aligned}b_1(\text{new}) &= b_1(\text{old}) + \alpha(-1 - z_{in_1}) \\&= .3 + (.5)(-1.55) \\&= -.475\end{aligned}$$

$$\begin{aligned}w_{11}(\text{new}) &= w_{11}(\text{old}) + \alpha(-1 - z_{in_1})x_1 \\&= .05 + (.5)(-1.55) \\&= -.725\end{aligned}$$

$$\begin{aligned}w_{21}(\text{new}) &= w_{21}(\text{old}) + \alpha(-1 - z_{in_1})x_2 \\&= .2 + (.5)(-1.55) \\&= -.575\end{aligned}$$

Example 2-20



update the weights on unit Z_2 as follows:

$$\begin{aligned}b_2(\text{new}) &= b_2(\text{old}) + \alpha(-1 - z_{in_2}) \\&= .15 + (.5)(-1.45) \\&= -.575\end{aligned}$$

$$\begin{aligned}w_{12}(\text{new}) &= w_{12}(\text{old}) + \alpha(-1 - z_{in_2})x_1 \\&= .1 + (.5)(-1.45) \\&= -.625\end{aligned}$$

$$\begin{aligned}w_{22}(\text{new}) &= w_{22}(\text{old}) + \alpha(-1 - z_{in_2})x_2 \\&= .2 + (.5)(-1.45) \\&= -.525\end{aligned}$$

Example 2-20

- After fourth epoch, the final weights are:

$$w_{11} = -0.73 \quad w_{12} = 1.27$$

$$w_{21} = 1.53 \quad w_{22} = -1.33$$

$$b_1 = -0.99 \quad b_2 = -1.09$$

Example 2-21

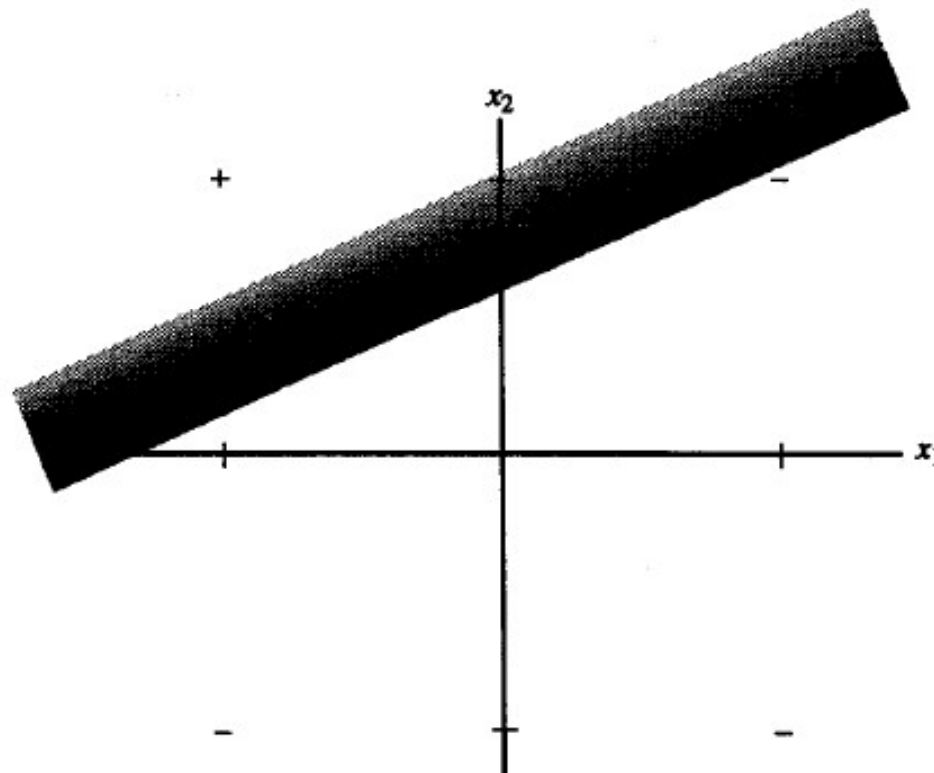
- ❑ Geometric interpretation of MADALINE weights.
- ❑ The positive response region for the Madaline trained in the previous example is the union of the regions where each of the hidden units have a positive response.
- ❑ For hidden unit Z1, the boundary line is:

$$x_2 = - \frac{w_{11}}{w_{21}} x_1 - \frac{b_1}{w_{21}}$$

$$= \frac{0.73}{1.53} x_1 + \frac{0.99}{1.53}$$

$$= 0.48 x_1 + 0.65$$

Example 2-21



Example 2-21

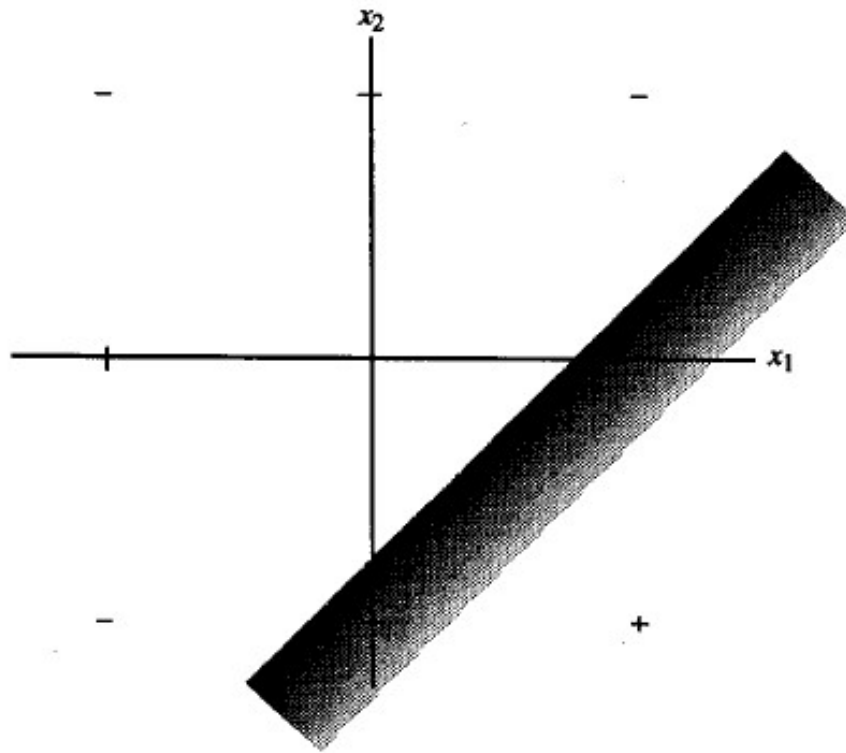
□ For hidden unit Z2, the boundary line is:

$$x_2 = -\frac{w_{12}}{w_{22}}x_1 - \frac{b_2}{w_{22}}$$

$$= \frac{1.27}{1.33}x_1 + \frac{1.09}{1.33}$$

$$= 0.96x_1 - 0.82$$

Example 2-21



Example 2-21

- The response of MADALINE is:

